

175. Physical Coding Sublayer (PCS), type 1.6TBASE-R

175.1 Overview

175.1.1 Scope

This clause specifies the Physical Coding Sublayer (PCS) that is common to a family of Physical Layer implementations known as 1.6TBASE-R. The 1.6TBASE-R PCS is a sublayer of the 1.6 Tb/s PHYs listed in Table 174–1. The term 1.6TBASE-R is used when referring generally to Physical Layers using the PCS defined in this clause. Annex 175A provides examples of RS-FEC codewords constructed with the method specified in this clause.

The 1.6TBASE-R PCS borrows heavily from the 400GBASE-R PCS defined in Clause 119 and the 800GBASE-R PCS defined in Clause 172.

175.1.2 Relationship of the 1.6TBASE-R PCS to other standards

Figure 175–1 shows the relationship of the 1.6TBASE-R PCS, along with other Ethernet sublayers, to the ISO Open System Interconnection (OSI) reference model.

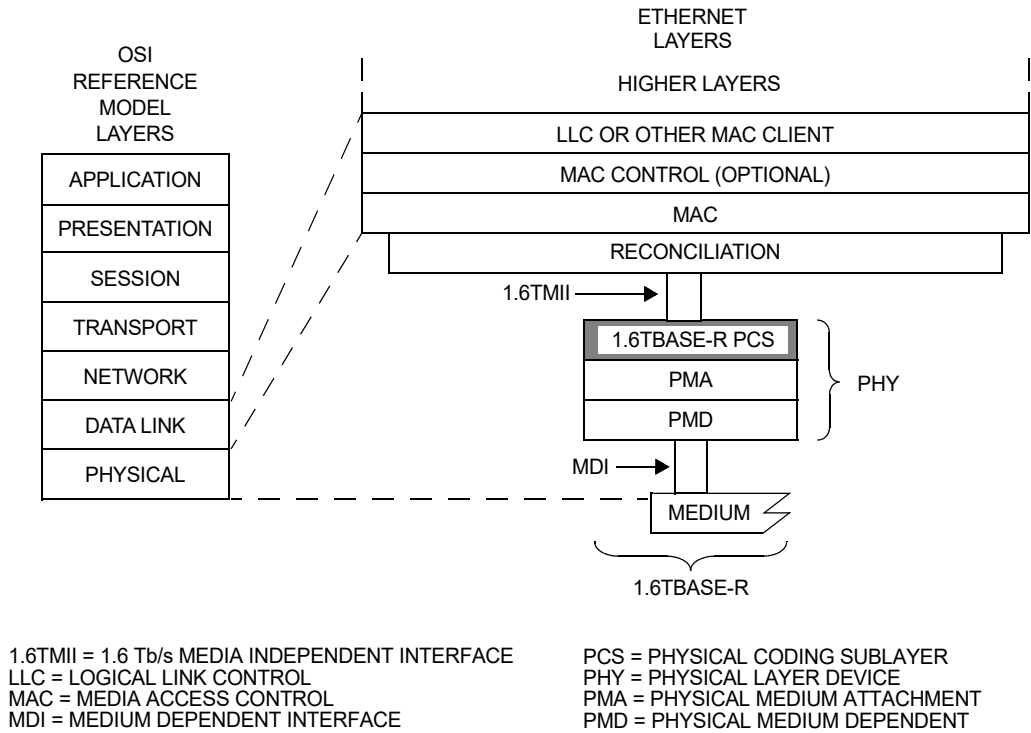


Figure 175–1—1.6TBASE-R PCS relationship to the ISO/IEC Open Systems Interconnection (OSI) reference model and IEEE 802.3 Ethernet model

175.1.3 Summary of functions

The 1.6TBASE-R PCS provides all services required by the MAC and RS, including the following:

- Encoding (decoding) of eight 1.6TMII data octets to (from) 66-bit blocks (64B/66B)
- Transcoding from (to) 66-bit blocks to (from) 257-bit blocks (256B/257B)
- Scrambling (descrambling) the 257-bit transcoded blocks
- Alignment marker (AM) insertion (removal)
- Reed-Solomon encoding (decoding) the scrambled 257-bit blocks
- FEC degrade signaling (required) and detection (optional)
- Distribution of Reed-Solomon encoded data to (from) PCS lanes
- Alignment marker lock, lane deskew and reordering of received PCS lanes
- Transferring encoded data to (from) the sublayer below the PCS
- Compensation for any rate differences caused by the insertion or deletion of alignment markers or due to any rate difference between the 1.6TMII and the sublayer below the PCS through the insertion or deletion of idle control characters

175.1.4 Inter-sublayer interfaces

The upper interface of the PCS connects to the Reconciliation Sublayer or a PHY 1.6TXS, through the PCS service interface. The lower interface of the PCS connects to a PMA or Inner FEC through the service interface below the PCS as necessary to support a PMD. The lower interface of the 1.6TBASE-R PCS has a nominal rate of 106.25 Gb/s on each of 16 PCS lanes, which provides capacity for the MAC data rate of 1.6 Tb/s.

While this specification defines interfaces in terms of bits, octets, and frames, implementations may choose other data-path widths for implementation convenience.

175.1.4.1 PCS service interface

The PCS service interface allows the 1.6TBASE-R PCS to transfer information to and from a PCS client as illustrated at the top of Figure 175–2.

When the client sublayer is the Reconciliation Sublayer, the PCS service interface is the 1.6 Tb/s Media Independent Interface (1.6TMII) defined in Clause 170.

When the client sublayer is the PHY 1.6TXS (see Figure 171–1), the PCS service interface is the 1.6TMII with additional signals PCS_status, TXRD, TXLD, RXRD, RXLD, TXLS and RXLS.

The PCS_status signal indicates the state of the PCS_status variable (see 175.2.6.2.2).

The TXRD, TXLD, RXRD, and RXLD signals support the FEC degrade function (see 175.3). The TXRD and TXLD signals indicate the state of the remote degrade and local degrade, respectively, as detected by the PHY 1.6TXS in the transmit direction (see 171.3.3a). The RXRD and RXLD signals indicate the state of the remote degrade and local degrade in the receive direction and are generated as follows:

- RXRD is set to the state of the rx_rm_degraded variable (see 175.2.5.5).
- RXLD is set to the logical OR of the FEC_degraded_SER variable (see 175.2.5.3) and the rx_local_degraded variable (see 175.2.5.5)

The TXLS and RXLS signals support passing link status information which would otherwise be handled by the IS_SIGNAL.request and IS_SIGNAL.indication primitives on an inter-sublayer service interface as described in 174.3. The TXLS signal indicates the state of the transmit link status as generated by the PHY

1.6TXS in the transmit direction (see 171.3.3a). The RXLS signal takes the value of the rx_link_status variable in the receive direction.

The rx_link_status variable shall take one of the values of FAIL, IN_PROGRESS, READY, or OK as defined in Table 175–1.

Table 175–1—rx_link_status generation

<i>inst:IS_SIGNAL.indication</i> ^a SIGNAL_OK	align_status ^b	rx_link_status
OK	true	OK
OK	false	READY
READY	don't care	READY
IN_PROGRESS	don't care	IN_PROGRESS
FAIL	don't care	FAIL

^a From the sublayer below the PCS.

^b See 175.2.6.2.2

175.1.4.2 Service interface below the PCS

The service interface below the PCS supports the exchange of encoded data between the PCS and the sublayer below the PCS. It is an instance of the inter-sublayer service interface defined in 174.3 and is described in an abstract manner which does not imply any particular implementation.

In the receive direction, the PCS receives 16 parallel streams of data via the *inst:IS_UNITDATA_0:15.indication* primitive and signal status information via the *inst:IS_SIGNAL.indication* primitive.

In the transmit direction, the PCS transmits 16 parallel streams of data via the *inst:IS_UNITDATA_0:15.request* primitive and provides signal status information via the *inst:IS_SIGNAL.request* primitive. When the interface above the PCS is the Reconciliation Sublayer, the SIGNAL_OK parameter of the *inst:IS_SIGNAL.request* primitive is set to OK if the *inst:IS_UNITDATA_0:15.request* data units are valid and is set to FAIL otherwise. When the interface above the PCS is the PHY 1.6TXS, the SIGNAL_OK parameter at the interface below the PCS is set according to Table 175–2.

Table 175–2—inst:IS_SIGNAL.request(SIGNAL_OK) generation

TXLS	<i>inst:IS_SIGNAL.request</i> SIGNAL_OK
OK	OK
READY	READY
IN_PROGRESS	IN_PROGRESS
FAIL	FAIL

175.1.5 Functional block diagram

Figure 175–2 provides a functional block diagram of the 1.6TBASE-R PCS and its primary interfaces.

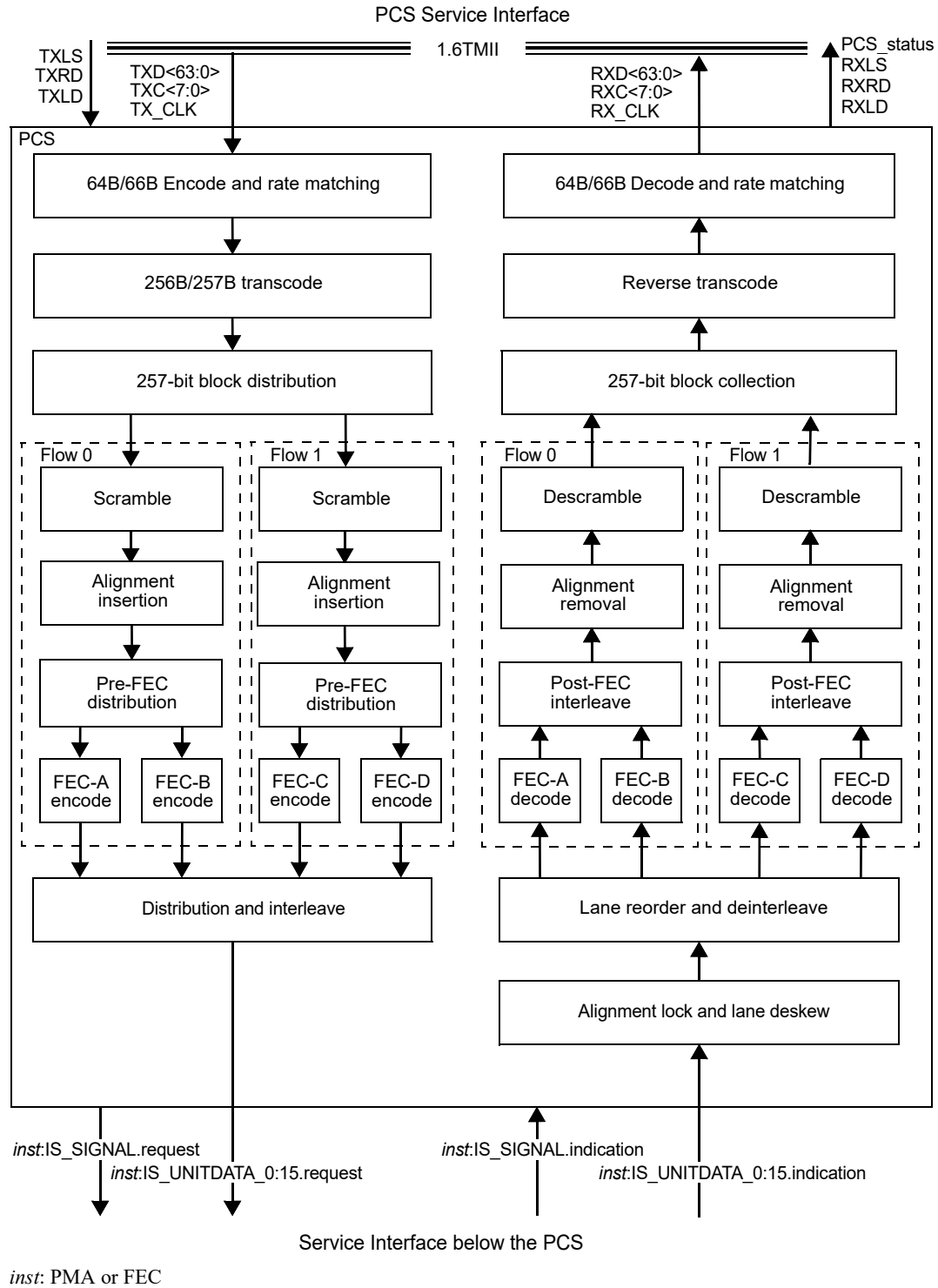


Figure 175–2—Functional block diagram

175.2 PCS functions

175.2.1 Overview

The 1.6TBASE-R PCS is composed of the PCS Transmit and PCS Receive processes. The PCS shields the Reconciliation Sublayer and MAC from the specific nature of the underlying channel.

When communicating with the 1.6TMII, the PCS uses an eight octet-wide, synchronous data path, with packet delineation being provided by transmit control signals ($\text{TXC}_{<n>} = 1$) and receive control signals ($\text{RXC}_{<n>} = 1$). When communicating with the PMA or Inner FEC sublayer, the 1.6TBASE-R PCS uses 16 encoded bit streams also known as PCS lanes. Per direction (receive or transmit), the PCS lanes originate from a common clock but may vary in phase and skew dynamically. The PMA and Inner FEC operate independently of packet boundaries. The PCS provides the functions necessary to map packets between the 1.6TMII format and the service interface below the PCS.

The PCS transmit function operates in normal mode or test-pattern mode.

When the transmit function is in normal mode, the PCS Transmit process continuously generates 66-bit blocks based on the $\text{TXD}_{<63:0>}$ and $\text{TXC}_{<7:0>}$ signals on the 1.6TMII. This takes place in the encode and rate matching function shown in Figure 175–2. The 66-bit blocks are transcoded to 257-bit blocks to reduce the line coding overhead. The 257-bit blocks are then distributed between two parallel transmit data flows, referred to as flow 0 and flow 1. Within each flow, the 257-bit blocks are first scrambled. Alignment markers are periodically inserted into the data stream of each flow following the requirements specified in 175.2.4.6. Within each flow, the data stream is distributed to two 5140-bit blocks and then FEC encoded. After FEC encoding, four codewords, one from each of the FEC encoders in each flow, are then interleaved and distributed to individual PCS lanes.

Transmit data units are sent to the service interface via the *inst:IS_UNITDATA_0:15.request* primitive.

When the transmit function is in test-pattern mode, a test pattern is packed into the transmit data units that are sent to the service interface below the PCS via the *inst:IS_UNITDATA_0:15.request* primitive.

In the receive direction, the PCS Synchronization process continuously monitors *inst:IS_SIGNAL.indication(SIGNAL_OK)*. When *SIGNAL_OK* indicates OK, the PCS synchronization process accepts data units via the *inst:IS_UNITDATA_0:15.indication* primitive. It attains alignment marker lock and deskews the PCS lanes based on the common marker (CM) portion of the alignment markers that are periodically transmitted on every PCS lane. Once all the PCS lanes are identified and re-ordered using the unique marker (UM) portion of the alignment markers, the *align_status* flag is set to true. Note that a particular transmit PCS lane might be received on any receive lane of the service interface due to the skew and multiplexing that occurs in the path.

The reordered PCS lanes are deinterleaved into two data flows, flow 0 and flow 1. Each data flow input consists of two FEC data streams. Within each flow, received data is processed by the FEC decoders, and re-interleaved on a 10-bit basis to form a data stream of 257-bit blocks. The alignment markers are removed, and the data is descrambled. After the descramble function, 257-bit blocks are collected from each flow and merged to form a single data flow. The 257-bit blocks are then reverse transcoded back to 66-bit blocks which are then 64B/66B decoded.

175.2.2 Use of 66-bit blocks

The PCS maps 1.6TMII signals into 66-bit blocks, and vice versa, using a 64B/66B coding scheme. The PCS functions ENCODE and DECODE generate, manipulate, and interpret 66-bit blocks as defined in 175.2.3.

175.2.3 64B/66B code

The 64B/66B code is identical to that specified in 119.2.3 with the following exceptions:

- 200GMII/400GMII is replaced with 1.6TMII.
- 200GBASE-R/400GBASE-R is replaced with 1.6TBASE-R.
- EEE and low power idle are not supported.

175.2.4 Transmit function

175.2.4.1 64B/66B encoder

The transmit PCS generates 66-bit blocks based on the TXD<63:0> and TXC<7:0> signals received on the 1.6TMII. Each 1.6TMII transfer is encoded into one 66-bit block. The contents of each 66-bit block are contained in a vector tx_coded<65:0> with tx_coded<1:0> containing the sync header and the remainder of the bits, the payload.

The transmit PCS may use either the state-diagram encoder defined in 119.2.4.1.1 or the stateless encoder defined in 119.2.4.1.2.

175.2.4.2 Rate matching

The transmit PCS may remove idle control characters or sequence ordered sets to compensate for the insertion of alignment markers. The transmit PCS may remove idle control characters or sequence ordered sets or may insert idle control characters to compensate for different clock domains on the 1.6TMII and the service interface below the PCS. See 82.2.3.6 and 82.2.3.9 for the deletion and insertion rules.

175.2.4.3 64B/66B to 256B/257B transcoder

The 64B/66B to 256B/257B transcoder is identical to that specified in 119.2.4.2.

Four consecutive tx_coded<65:0> blocks are used to construct a 257-bit block, tx_xcoded<256:0>.

NOTE—The stream of 257-bit blocks generated by this process, together with the FEC_degraded_SER and rx_local_degraded variables should be used as the reference signal for mapping to OTN.

175.2.4.4 Block distribution

The 257-bit blocks, tx_xcoded<256:0>, from the transcoder are distributed to the two flows in an alternating fashion by the block distribution function such that one 257-bit block is sent to flow 0, the next 257-bit block is sent to flow 1, the following 257-bit block is sent to flow 0, and subsequent 257-bit blocks continue the distribution procedure across the two flows. This creates two streams of transcoded blocks, tx_xcoded_f0<256:0> to flow 0, and tx_xcoded_f1<256:0> to flow 1.

175.2.4.5 Scrambler

The transcoded 257-bit blocks, tx_xcoded_f0<256:0> in flow 0 and tx_xcoded_f1<256:0> in flow 1, are scrambled with a self-synchronizing scrambler within each flow to generate tx_scrambled_f0<256:0> and tx_scrambled_f1<256:0>, respectively. The scrambler is identical to the scrambler in 49.2.6 using the polynomial in Equation (49–1). When reset is asserted, the two scramblers shall be initialized to non-zero values different from each other.

175.2.4.6 Alignment marker mapping and insertion

In order to support deskew and reordering of the individual PCS lanes at the receive PCS, alignment markers (AMs) corresponding to PCS lanes are periodically inserted after being processed by the alignment marker mapping function. The alignment marker mapping function compensates for the operation of the symbol distribution function and rearranges the alignment marker bits so that they appear on the PCS lanes intact and in the desired sequence. This preserves the properties of the alignment markers (e.g. DC balance, transition density) and provides a deterministic pattern for the purpose of synchronization.

An alignment marker group is composed of the alignment markers for all 16 PCS lanes plus an additional 133-bit pad and a 3-bit status field to yield the equivalent of eight 257-bit blocks. The alignment marker group is divided evenly between the two flows and is aligned to the beginning of four FEC messages across both flows. The alignment marker group interrupts any data transfer that is already in progress. The pad bits at the end of the alignment marker group shall be set to a free-running PRBS9 pattern in each flow, defined by the polynomial $x^9 + x^5 + 1$. The initial value of the PRBS9 pattern generators after PCS reset is deasserted may be any pattern other than all zeros. The 3-bit transmit alignment marker status field (tx_am_sf) carries local and remote FEC degrade status using an end-to-end process described in 116.6 and is appended to the padding in flow 1.

Idle control characters or sequence ordered sets are removed, if necessary, to accommodate the insertion of the alignment markers. The alignment marker group is not scrambled which allows the receiver to directly search for and find the individual alignment markers, deskew the PCS lanes, and reassemble the aggregate stream before FEC decoding is performed. The alignment markers are formed from a known pattern that is defined to be balanced and with many transitions and therefore scrambling is not necessary.

The format of the alignment marker in each PCS lane is shown in Figure 175–3. There is a portion that is common across all alignment markers (designated as CM₀ to CM₅), a unique portion per PCS lane (designated as UM₀ to UM₅), and a unique pad per PCS lane (designated as UP₀ to UP₂). Common synchronization logic independent of the received PCS lane number may be used with the common portion of the alignment marker. The unique pad (UP₀ to UP₂) within the alignment markers and the PRBS9 pad at the end of the alignment marker group are ignored on receive.

The format shown in Figure 175–3 defines how the alignment markers appear on a given PCS lane. In the FEC codewords, they appear in a permuted format due to the codeword interleaving that occurs before FEC codewords are distributed to PCS lanes.

The 3-bit transmit alignment marker status field, tx_am_sf, allows the local PCS to communicate the status of the FEC degraded feature to the remote PCS and is assigned values as shown in Table 175–3.

Table 175–3—tx_am_sf field assignments

Bit	Field name	Assignment when the client sublayer is the RS	Assignment when the client sublayer is the PHY 1.6TXS
<2>	remote degraded	FEC_degraded_SER + rx_local_degraded	TXRD
<1>	local degraded	0	TXLD
<0>	<reserved for future use>	0	0

In Table 175–3, the PCS status variable FEC_degraded_SER is defined in 175.2.5.3. The rx_local_degraded variable is defined in 175.2.5.5. TXRD and TXLD are signals from the PHY 1.6TXS on the PCS service interface (see 175.1.4.1), and + denotes logical OR.

175.2.4.6.1 Alignment marker creation

The alignment marker mapping function creates a set of 16 alignment markers, and in combination with an additional 133-bit PRBS9 pad and 3-bit status field, the PCS generates an alignment marker group.

Let $am_x<119:0>$ be the alignment marker for PCS lane x , $x=0$ to 15, where bit 0 is the first bit transmitted.

$am_x<119:0>$ is constructed as shown in Figure 175–3 using the values in Table 175–4 for each PCS lane number x .

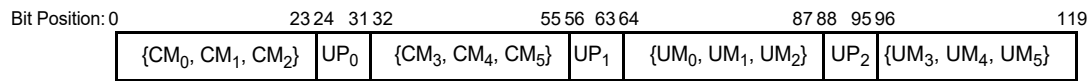


Figure 175–3—Alignment marker format

Table 175–4—1.6TBASE-R alignment marker encodings

PCS lane	Encoding ^a														
	CM0	CM1	CM2	UP0	CM3	CM4	CM5	UP1	UM0	UM1	UM2	UP2	UM3	UM4	UM5
0	0x9A	0x4A	0x26	0xB6	0x65	0xB5	0xD9	0xD9	0xFE	0x8E	0x0C	0x26	0x01	0x71	0xF3
1				0x04				0x67	0xA5	0x21	0x81	0x98	0x5A	0xDE	0x7E
2				0x46				0xFE	0xC1	0x0C	0xA9	0x01	0x3E	0xF3	0x56
3				0x5A				0x84	0x79	0x7F	0x2F	0x7B	0x86	0x80	0xD0
4				0xE1				0x19	0xD5	0xAE	0x0D	0xE6	0x2A	0x51	0xF2
5				0xF2				0x4E	0xED	0xB0	0x2E	0xB1	0x12	0x4F	0xD1
6				0x3D				0xEE	0xBD	0x63	0x5E	0x11	0x42	0x9C	0xA1
7				0x22				0x32	0x29	0x89	0xA4	0xCD	0xD6	0x76	0x5B
8				0x60				0x9F	0x1E	0x8C	0x8A	0x60	0xE1	0x73	0x75
9				0x6B				0xA2	0x8E	0x3B	0xC3	0x5D	0x71	0xC4	0x3C
10				0xFA				0x04	0x6A	0x14	0x27	0xFB	0x95	0xEB	0xD8
11				0x6C				0x71	0xDD	0x99	0xC7	0x8E	0x22	0x66	0x38
12				0x18				0x5B	0x5D	0x09	0x6A	0xA4	0xA2	0xF6	0x95
13				0x14				0xCC	0xCE	0x68	0x3C	0x33	0x31	0x97	0xC3
14				0xD0				0xB1	0x35	0x04	0x59	0x4E	0xCA	0xFB	0xA6
15				0xB4				0x56	0x59	0x45	0x86	0xA9	0xA6	0xBA	0x79

^a Each octet is transmitted LSB to MSB.

As an example, the variable `am_0` is sent as (left most bit sent first, showing the first 32 bits transmitted of `am_0`):

01011001 01010010 01100100 01101101

NOTE—A text file containing the alignment marker values shown in Table 175–4 is available at <https://standards.ieee.org/downloads/802.3>.

Editor's note (to be removed prior to publication): The following file will be placed in the 802.3 downloads folder referenced above when this amendment is published:
https://www.ieee802.org/3/dj/public/24_05/opsasnick_3dj_02_2405.txt

The alignment marker group is divided between the two flows and mapped into two variables, `am_mapped_f0` and `am_mapped_f1`. The variable `am_mapped_f0` is inserted in flow 0 which produces FEC codewords A and B, and `am_mapped_f1` is inserted in flow 1 which produces FEC codewords C and D. The 16 alignment markers, `am_x`, are mapped into these two variables using the following procedure to compensate for the pre-FEC and post-FEC symbol distribution so that the AMs appear on the PCS lanes in the correct sequence:

For all $k=0$ to 2

For all $j=0$ to 15

$\text{am_mapped_f0} \langle (320k+20j+9) : (320k+20j) \rangle = \text{am_}\{j\} \langle (40k+9) : (40k) \rangle$; CW A
 $\text{am_mapped_f0} \langle (320k+20j+19) : (320k+20j+10) \rangle = \text{am_}\{j\} \langle (40k+19) : (40k+10) \rangle$; CW B
 $\text{am_mapped_f1} \langle (320k+20j+9) : (320k+20j) \rangle = \text{am_}\{j\} \langle (40k+29) : (40k+20) \rangle$; CW C
 $\text{am_mapped_f1} \langle (320k+20j+19) : (320k+20j+10) \rangle = \text{am_}\{j\} \langle (40k+39) : (40k+30) \rangle$; CW D

The additional 133-bit pad is then appended to the variables `am_mapped_f0` and `am_mapped_f1`. In the following expressions, `PRBS9_f0` and `PRBS9_f1` are outputs of the PRBS pattern generators in flow 0 and flow 1, respectively.

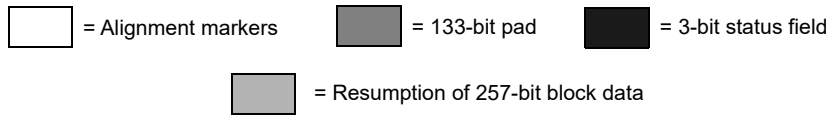
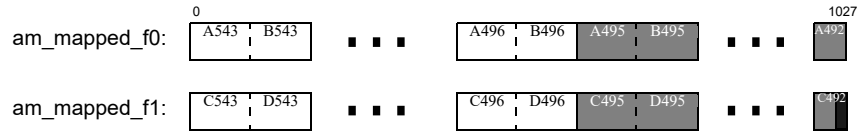
$\text{am_mapped_f0} \langle 1027:960 \rangle = \text{PRBS9_f0} \langle 67:0 \rangle$
 $\text{am_mapped_f1} \langle 1024:960 \rangle = \text{PRBS9_f1} \langle 64:0 \rangle$

The 3-bit transmit alignment marker status field is then appended to the variable `am_mapped_f1` as follows:

$\text{am_mapped_f1} \langle 1027:1025 \rangle = \text{tx_am_sf} \langle 2:0 \rangle$

The alignment marker mappings along with the padding and tx_am_sf fields are shown in Figure 175–4. Note that am_mapped_f0 contains 10-bit symbols of FEC codewords A and B, and am_mapped_f1 contains 10-bit symbols of FEC codewords C and D.

PCS lane	10-bit symbol index															
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	A543 0	B543	C543	D543	A527	B527 am' 0	C527	D527	A511	B511	C511	D511 119	A 495	B495	C495	D495
1	A	B	C	D	A	B am' 1	C	D	A	B	C	D	A	B	C	D
2	A	B	C	D	A	B am' 2	C	D	A	B	C	D	A	B	C	D
3	A	B	C	D	A	B am' 3	C	D	A	B	C	D	A492	B	C492	D
4	A	B	C	D	A	B am' 4	C	D	A	B	C	D	A	B	C	D
5	A	B	C	D	A	B am' 5	C	D	A	B	C	D	A	B	C	D
6	A	B	C	D	A	B am' 6	C	D	A	B	C	D	A	B	C	D
7	A	B	C	D	A	B am' 7	C	D	A	B	C	D	A	B	C	D
8	A	B	C	D	A	B am' 8	C	D	A	B	C	D	A	B	C	D
9	A	B	C	D	A	B am' 9	C	D	A	B	C	D	A	B	C	D
10	A	B	C	D	A	B am' 10	C	D	A	B	C	D	A	B	C	D
11	A	B	C	D	A	B am' 11	C	D	A	B	C	D	A	B	C	D
12	A	B	C	D	A	B am' 12	C	D	A	B	C	D	A	B	C	D
13	A	B	C	D	A	B am' 13	C	D	A	B	C	D	A	B	C	D
14	A	B	C	D	A	B am' 14	C	D	A	B	C	D	A	B	C	D
15	A528	B528	C528	D528	A512	B512 am' 15	C512	D512	A496	B496	C496	D496	A	B	C	D



A = from FEC codeword A B = from FEC codeword B C = from FEC codeword C D = from FEC codeword D

Figure 175–4—1.6TBASE-R alignment marker mapping to PCS lanes

175.2.4.6.2 Alignment marker insertion

The alignment marker group shall be inserted so it appears in the output stream every 655360×257 -bit blocks (equivalent to 32768 codewords). The alignment marker group is divided evenly between the two flows such that am_mapped_f0<1027:0> and am_mapped_f1<1027:0> are inserted into flow 0 and flow 1 every 327680×257 -bit blocks.

In each flow, a 10280-bit block of data (equivalent to two codewords) is formed and represented by the variable tx_scrambled_am_f0<10279:0> in flow 0 and tx_scrambled_am_f1<10279:0> in flow 1. These variables are constructed in one of two ways, with or without alignment markers. With AMs, the block in each flow contains one-half of the alignment marker group followed by 36×257 -bit blocks of scrambled data. Without AMs, each block contains 40×257 -bit blocks of scrambled data.

Let the vector $\text{tx_scrambled_f0}_{i<256:0>}$ represent consecutive values of $\text{tx_scrambled_f0}_{<256:0>}$, and let $\text{tx_scrambled_f1}_{i<256:0>}$ represent consecutive values of $\text{tx_scrambled_f1}_{<256:0>}$.

For the 10280-bit blocks with an alignment marker group inserted:

$$\text{tx_scrambled_am_f0}_{<1027:0>} = \text{am_mapped_f0}_{<1027:0>}$$

$$\text{tx_scrambled_am_f1}_{<1027:0>} = \text{am_mapped_f1}_{<1027:0>}$$

For all $i=0$ to 35

$$\text{tx_scrambled_am_f0}_{<(257i+1284):(257i+1028)>} = \text{tx_scrambled_f0}_{i<256:0>}$$

$$\text{tx_scrambled_am_f1}_{<(257i+1284):(257i+1028)>} = \text{tx_scrambled_f1}_{i<256:0>}$$

For the 10280-bit blocks without an alignment marker group:

For all $i=0$ to 39

$$\text{tx_scrambled_am_f0}_{<(257i+256):(257i)>} = \text{tx_scrambled_f0}_{i<256:0>}$$

$$\text{tx_scrambled_am_f1}_{<(257i+256):(257i)>} = \text{tx_scrambled_f1}_{i<256:0>}$$

Alignment marker repetition rate within each flow is shown in Figure 175–5.

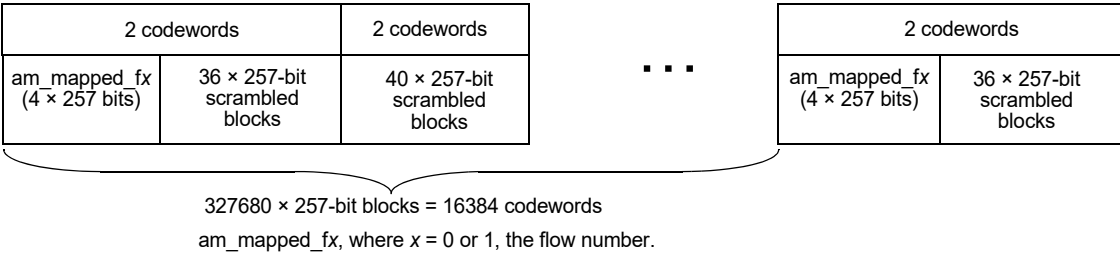


Figure 175–5—1.6TBASE-R alignment marker insertion period (per-flow)

The alignment marker insertion within each flow shall occur at the same point relative to the original stream of 257-bit blocks before block distribution. This requirement is illustrated in Figure 175–6 where the alignment marker group is inserted after block $k-1$ and prior block k .

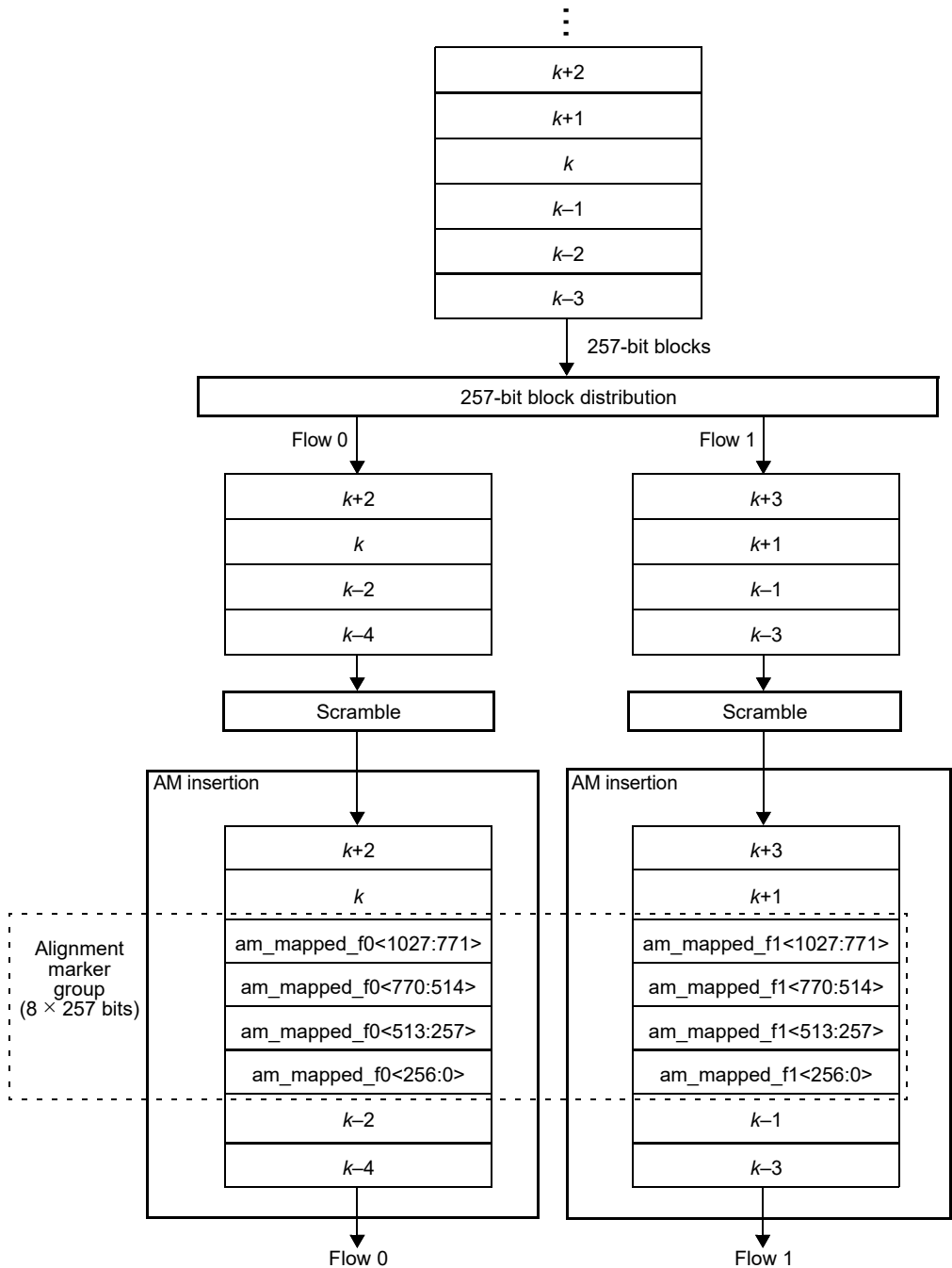


Figure 175–6—1.6TBASE-R PCS alignment marker insertion

175.2.4.7 Pre-FEC distribution

In order to improve error correction capability, four Reed-Solomon FEC codewords are interleaved when forming the PCS lanes. To enable this interleaving, a pre-FEC distribution function occurs in each flow. The pre-FEC distribution function receives a 10280-bit block and performs a 10-bit symbol round-robin distribution to form two FEC messages. In flow 0, m_A and m_B are formed from $tx_scrambled_am_f0$. In flow 1, m_C and m_D are formed from $tx_scrambled_am_f1$. Each resulting FEC message consists of 514 10-bit symbols. The following procedure describes the 10-bit round-robin distribution process.

For all $i=0$ to 513

$$\begin{aligned} m_A\langle(513-i)\rangle\langle 9:0\rangle &= tx_scrambled_am_f0\langle(20i+9) : (20i)\rangle \\ m_B\langle(513-i)\rangle\langle 9:0\rangle &= tx_scrambled_am_f0\langle(20i+19) : (20i+10)\rangle \\ m_C\langle(513-i)\rangle\langle 9:0\rangle &= tx_scrambled_am_f1\langle(20i+9) : (20i)\rangle \\ m_D\langle(513-i)\rangle\langle 9:0\rangle &= tx_scrambled_am_f1\langle(20i+19) : (20i+10)\rangle \end{aligned}$$

Where $m_X\langle 513\rangle$ is the first transmitted symbol and $m_X\langle 0\rangle$ is the last transmitted symbol of each message, and $bit\langle 0\rangle$ is the first transmitted bit and $bit\langle 9\rangle$ is the last transmitted bit of each symbol.

175.2.4.8 Reed-Solomon encoder

The Reed-Solomon encoder in each flow is identical to that specified in 119.2.4.6. In flow 0, two 5140-bit message blocks, m_A and m_B , are encoded using the RS(544,514) encoder into codeword A (c_A) and codeword B (c_B), respectively. In flow 1, two 5140-bit message blocks, m_C and m_D , are encoded using the RS(544,514) encoder into codeword C (c_C) and codeword D (c_D), respectively.

Examples of RS-FEC codewords are provided in Annex 175A.

175.2.4.9 Symbol distribution

After RS-FEC encoding, the four codewords across both flows are interleaved on a 10-bit symbol basis to form 16 PCS lanes as shown in Figure 175–7. Each PCS lane alternates 10-bit FEC symbols from the four codewords in the repeating pattern, {A, B, C, D, A, B, C, D, ...}.

Let $pcsl_i\langle 1359:0\rangle$ represent the 1360 bits transmitted on PCS lane i for each set of four codewords where $bit\langle 0\rangle$ is transmitted first on each PCS lane.

The interleaving of the four codewords follows this procedure:

For all $j=0$ to 33

For all $i=0$ to 15

$$\begin{aligned} pcsl_i\langle(40j+9) : (40j)\rangle &= c_A\langle 543-16j-i\rangle\langle 9:0\rangle \\ pcsl_i\langle(40j+19) : (40j+10)\rangle &= c_B\langle 543-16j-i\rangle\langle 9:0\rangle \\ pcsl_i\langle(40j+29) : (40j+20)\rangle &= c_C\langle 543-16j-i\rangle\langle 9:0\rangle \\ pcsl_i\langle(40j+39) : (40j+30)\rangle &= c_D\langle 543-16j-i\rangle\langle 9:0\rangle \end{aligned}$$

175.2.4.10 Transmit data

The 16 PCS lanes formed from the symbol distribution are transmitted on the service interface below the PCS. PCS lane i shall be transmitted on $inst:IS_UNITDATA_i.request$, for $i=0$ to 15.

The transmit data distribution and bit ordering are illustrated in Figure 175–7.

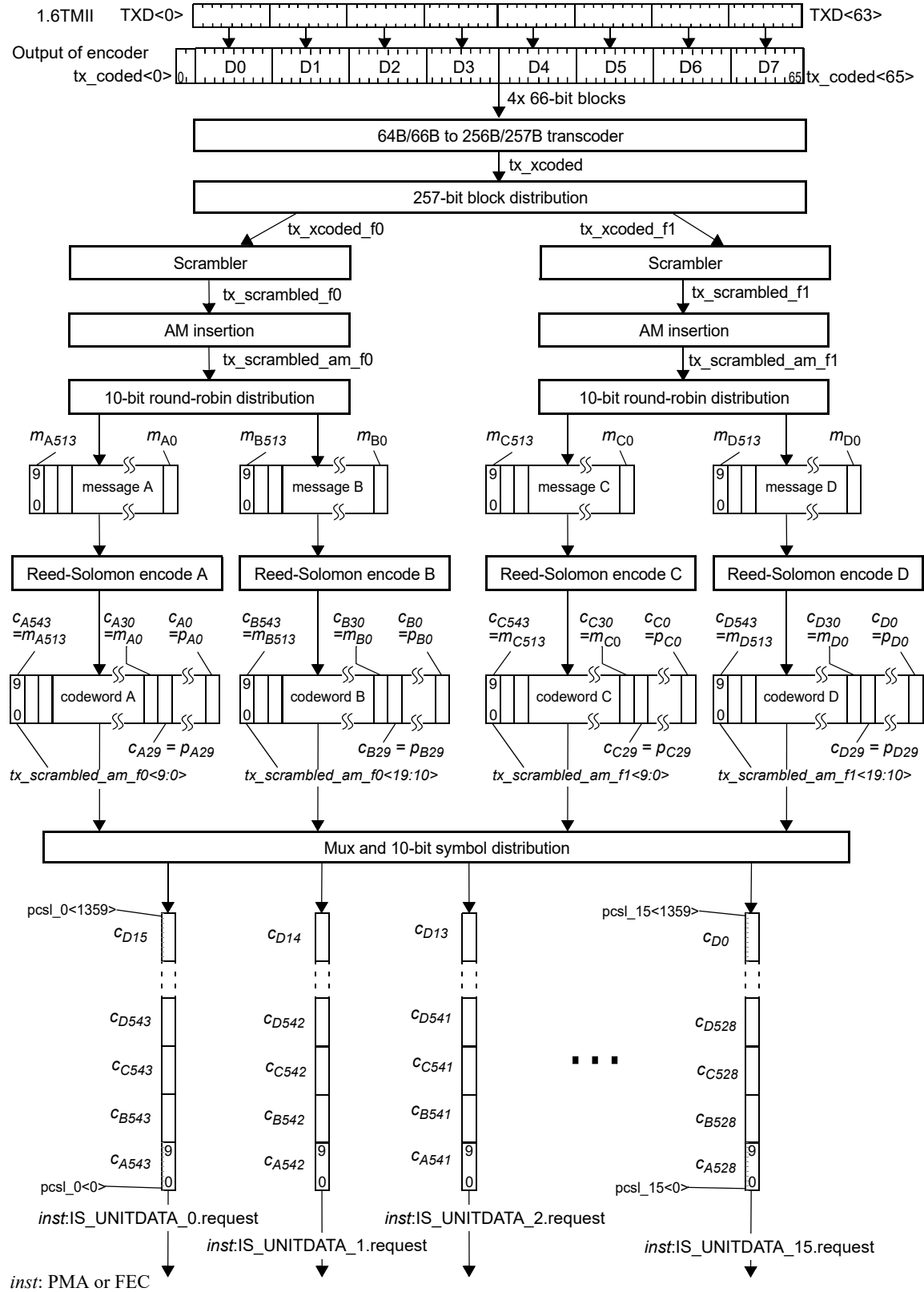


Figure 175-7—1.6TBASE-R PCS transmit data distribution and bit ordering

175.2.4.11 Test-pattern generator

The PCS shall have the ability to generate a scrambled idle test pattern which is suitable for receiver tests and for certain transmitter tests. When test pattern generation is enabled by the PCS control variable `tx_test_mode` (set to 1), the scrambled idle test pattern is generated by the PCS. The scrambled idle test pattern is the output of the PCS when the input to the PCS at the 1.6TmII is composed only of idle control characters.

175.2.5 Receive function

175.2.5.1 Alignment lock and deskew

The alignment lock and deskew is identical to that specified in 119.2.5.1 with $n = 16$ and the following exceptions:

- Figure 119–13 is replaced with Figure 175–8 and Figure 175–9.
- The maximum Skew is changed from 180 ns to 152 ns.

175.2.5.2 Lane reorder and deinterleave

The PCS lanes might be received in any order on the lanes of the service interface below the PCS. The PCS receive function shall order the PCS lanes according to the PCS lane number. The PCS lane number is defined by the unique portion (UM_0 to UM_5) of the alignment marker that is mapped to each PCS lane (see 175.2.4.6).

After all PCS lanes are aligned, deskewed, and reordered, they are deinterleaved to reconstruct the original four streams of FEC codewords.

175.2.5.3 Reed-Solomon decoder

The Reed-Solomon decoder is identical to that specified in 119.2.5.3, with the following exceptions:

- When `FEC_bypass_indication_enable` is asserted, the associated additional error monitoring counts the number of symbol errors detected in consecutive non-overlapping blocks of 8192 codewords across all four FEC decoders to determine whether to set `hi_ser`.
- When `FEC_degraded_SER_enable` is asserted, the associated additional error monitoring counts the number of symbol errors detected on all PCS lanes, where the least significant two bits of `FEC_degraded_SER_interval` are ignored (evaluated as “00”) to make the number of codewords a multiple of four.
- References to PCS management objects in 119.3 refer instead to 175.8.

The following counters shall be implemented:

FEC_corrected_cw_counter

A 32-bit counter that counts once for each corrected FEC codeword processed when `align_status` is true as set by the PCS synchronization process (see 175.2.6.3). A corrected FEC codeword is one that contains errors and was corrected by the Reed-Solomon decoder. Note that this counter counts all corrected codewords across all four FEC decoders.

FEC_uncorrected_cw_counter

A 32-bit counter that counts once for each uncorrected FEC codeword processed when `align_status` is true. An uncorrected FEC codeword is one that contains errors that were detected but not corrected by the Reed-Solomon decoder. Note that this counter counts all uncorrected codewords across all four FEC decoders.

FEC_symbol_error_counter_{*i*}

A set of sixteen 32-bit counters where counter *i* counts once for each 10-bit symbol corrected on PCS lane *i* when align_status is true (*i*=0 to 15). Note that symbols from each PCS lane are distributed across all four FEC codeword decoders.

FEC_cw_counter

A 48-bit counter that counts all FEC codewords received by the PCS when align_status is true.

FEC_codeword_error_bin_{*i*}

A set of fifteen 32-bit counters where counter *i* counts once for each codeword received with exactly *i* correctable 10-bit symbols when align_status is true (*i*=1 to 15). For example, if a codeword has exactly 5 errored 10-bit symbols, then FEC_codeword_error_bin_5 is incremented. Note that these counters count all codewords with correctable errors across all four FEC decoders.

175.2.5.4 Post-FEC interleave

In each flow, after the Reed-Solomon decoders process the codewords, the two resulting message blocks are interleaved on a 10-bit basis to form rx_scrambled_am, corresponding to 40 transcoded blocks, in order to recreate the transmitted data stream.

175.2.5.5 Alignment marker removal

In each flow, every 327680 × 257-bit blocks (corresponding to 16 384 codewords), the first 1028 bits of an rx_scrambled_am block is the vector am_rx<1027:0>, where bit 0 is the first bit received. The specific codewords that include this vector are indicated by the alignment lock and deskew function. The 3-bit receive alignment marker status field (rx_am_sf) is assigned from the variable am_rx in flow 1 as follows:

rx_am_sf<2:0> = am_rx<1027:1025> (only in flow 1)

rx_am_sf<2:0> is used to generate the following PCS status variables:

rx_local_degraded

Boolean variable that is set to true when the receiver detects rx_am_sf<1> asserted for two consecutive alignment marker periods. It is set to false when rx_am_sf<1> is deasserted for two consecutive alignment marker periods.

rx_rm_degraded

Boolean variable that is set to true when the receiver detects rx_am_sf<2> asserted for two consecutive alignment marker periods. It is set to false when rx_am_sf<2> is deasserted for two consecutive alignment marker periods.

The vector am_rx is removed from rx_scrambled_am to create rx_scrambled prior to descrambling.

175.2.5.6 Descrambler

The descrambler shall process rx_scrambled<256:0> to reverse the effect of the scrambler using the polynomial given in Equation (49–1). It shall produce the same result as the implementation shown in Figure 49–10.

175.2.5.7 Block collection

The block collection reverses the block distribution performed in the transmitter (see 175.2.4.4) by combining the 257-bit blocks from the two flows in an alternating fashion to form a single stream of 257-bit blocks. The first 257-bit block after the alignment marker group from flow 0 shall be followed by the first 257-bit block after the alignment marker group from flow 1.

NOTE—The stream of 257-bit blocks generated by this process should be used as the reference signal for demapping from OTN.

175.2.5.8 256B/257B to 64B/66B transcoder

The 256B/257B to 64B/66B transcoder in each flow is identical to that specified in 119.2.5.7.

175.2.5.9 64B/66B decoder

The receive PCS decodes 66-bit blocks to produce RXD<63:0> and RXC<7:0> for transmission to the 1.6TMII. One 1.6TMII transfer is decoded from each 66-bit block. The receive PCS may use either the state-diagram decoder defined in 119.2.5.8.1 or the stateless decoder defined in 119.2.5.8.2.

175.2.5.10 Rate matching

The receive PCS may insert idle control characters to compensate for the deletion of alignment markers. The receive PCS may remove idle control characters or sequence ordered sets or may insert idle control characters to compensate for different clock domains on the service interface below the PCS and the 1.6TMII. See 82.2.3.6 and 82.2.3.9 for the deletion and insertion rules.

NOTE—If no idle control characters are inserted to compensate for the removal of alignment markers, the average data rate on the 1.6TMII will be reduced by approximately 12 ppm.

175.2.6 State diagrams

This subclause is composed of state diagrams, including the associated definitions of constants, variables, functions, and counters.

175.2.6.1 State diagram conventions

The notation used in the state diagrams follows the conventions of 21.5.

Should there be a discrepancy between a state diagram and descriptive text, the state diagram prevails.

175.2.6.2 State diagram definitions

1.6TBASE-R PCS uses the following state diagrams defined in Clause 119:

- Figure 119–12, “Alignment marker lock state diagram”
- Figure 119–14, “Transmit state diagram”
- Figure 119–15, “Receive state diagram”

Any constants, variables, functions, or counters used in the above Clause 119 state diagrams which are not defined in the following subclauses shall use the values and definitions from 119.2.6.2.

175.2.6.2.1 Constants

The constants are the same as those specified in 119.2.6.2.1 with the following modification:

- 200GMII/400GMII is replaced with 1.6TMII.

175.2.6.2.2 Variables

align_status

Boolean variable that indicates when all PCS lanes are synchronized and aligned. The value of align_status is set by the PCS synchronization state diagram (Figure 175–8).

all_locked	1
Boolean variable that is set to true when amps_lock<x> is true for all x and is set to false when	2
amps_lock<x> is false for any x.	3
amps_lock<x>	4
Boolean variable that indicates the receiver has detected the location of the alignment marker	5
payload sequence for a given lane on the service interface below the PCS, where x=0 to 15 for	6
1.6TBASE-R. The value of amps_lock<x> is set by the alignment marker lock state diagram	7
(Figure 119–12).	8
current_pcsl	9
Same behavior as 119.2.6.2.2 with the exception that the unique marker values are defined in	10
Table 175–4.	11
current_pcsl	12
Same behavior as 119.2.6.2.2 with the exception that the unique marker values are defined in	13
Table 175–4.	14
cw_bad<z>	15
Boolean variable that is set to true if the Reed-Solomon decoder fails to correct the current FEC	16
codeword z and is set to false otherwise. z = {A, B, C, or D}.	17
deskew_done	18
Boolean variable that is set to true when pcs_enable_deskew is true and the deskew process is	19
completed. Otherwise, this variable is set to false.	20
deskew_failed	21
Boolean variable that indicates the PCS synchronization process failed to identify 16 unique PCS	22
lanes and is used to set the restart_lock variable. The value of deskew_failed is set by the PCS	23
synchronization state diagram (Figure 175–8).	24
hi_ser	25
A variable that is set to one when RS-FEC error indication is bypassed and a high number of FEC	26
symbols have errors within a block of codewords as described in 175.2.5.3 and 119.2.5.3, and is	27
zero otherwise.	28
pcs_alignment_valid	29
Boolean variable that is set to true if all PCS lanes are aligned. PCS lanes are considered to be	30
aligned when amps_lock<x> is true for all x, each PCS lane is locked to a unique alignment marker	31
payload sequence, and the PCS lanes are deskewed. Otherwise, this variable is set to false.	32
pcs_enable_deskew	33
Boolean variable that enables and disables the PCS synchronization process. Received bits may be	34
discarded whenever deskew is enabled. Its value is set by the PCS synchronization state diagram	35
(Figure 175–8).	36
pcs_lane_mapping<x>	37
A variable that holds the value of the PCS lane number received on the service interface below the	38
PCS, inst:IS_UNITDATA_x.indication. Its value is set by the alignment marker lock state diagram	39
(Figure 119–12).	40
PCS_reset	41
Boolean variable that is true when set by a management entity and is false otherwise.	42
PCS_status	43
Boolean variable that is true when align_status is true and is false otherwise.	44
reset	45
Boolean variable that controls the resetting of the PCS sublayer. It is true whenever a reset is	46
necessary including when PCS_reset is true and during power on, and is false otherwise.	47
restart_lock	48
Boolean variable that is used to restart the alignment marker lock process on all PCS lanes (see	49
	50
	51
	52
	53
	54

Figure 119–12). Its value is set to true if `deskew_failed` is true or if `three_bad_cw<z>` is true for any z . Otherwise, this variable is set to false.

`test_cw<z>`

Boolean variable that is set to true when a new FEC codeword z is available for decoding and is set to false by the PCS codeword monitor state diagram (Figure 175–9). $z = \{A, B, C, \text{ or } D\}$.

`three_bad_cw<z>`

Boolean variable that is set by the PCS codeword monitor state diagram (Figure 175–9) and is used to set the `restart_lock` variable. It indicates that three consecutive uncorrected codewords are received for FEC codeword z where $z = \{A, B, C, \text{ or } D\}$.

175.2.6.2.3 Functions

The functions are the same as those specified in 119.2.6.2.3 with the following modifications:

- 200GMII/400GMII is replaced with 1.6TMII.
- EEE and low power idle are not supported.

175.2.6.2.4 Counters

`amp_counter`

Counts the interval of 32768 FEC codewords containing alignment marker payload sequences for the 1.6TBASE-R PCS.

`cw_bad_count<z>`

Counts the number of consecutive uncorrected FEC codewords by FEC decoder z . This counter is set to zero when a FEC codeword z is received and `cw_bad<z>` is false. $z = \{A, B, C, \text{ or } D\}$.

175.2.6.3 State diagram figures

The 1.6TBASE-R PCS shall implement sixteen alignment marker lock processes as depicted in Figure 119–12 with these exceptions:

- The alignment marker values are used as specified in Table 175–4.
- `amp_counter` counts the interval of 32768 FEC codewords.

An alignment marker lock process operates independently on each input service lane. The alignment marker lock state diagram shown in Figure 119–12 determines when the PCS has obtained alignment marker lock to the received bit stream for a given lane of the service interface. Each alignment marker lock process looks for two valid alignment markers 1114112×10 -bit Reed-Solomon symbols apart to gain alignment marker lock. When the alignment marker lock process achieves lock for a lane, the variable `amps_lock<x>` is set to true ($x = 0$ to 15), and the PCS lane number received on that lane of the service interface is stored in the PCS status variable `pcs_lane_mapping<x>`. Once in lock, a lane goes out of alignment marker lock when `restart_lock` is signaled by the PCS synchronization process (when three uncorrectable codewords in a row are seen) or when the alignment marker lock process sees five alignment markers in a row that fail to match the expected pattern on a given lane.

The PCS shall run the synchronization process as depicted in Figure 175–8 and four independent codeword monitor processes as shown in Figure 175–9. The PCS synchronization process is responsible for determining if the PCS is capable of presenting coherent data to the 1.6TMII. The synchronization process ensures that all PCS lanes have alignment marker lock, are locked to different alignment markers, and that the skew is within the PCS deskew limit. When these conditions are met, the `align_status` variable is set to true. The synchronization process, along with the 16 alignment marker lock processes, are restarted if any of the four FEC decoder blocks process three consecutive uncorrectable codewords as indicated by the PCS codeword monitor processes.

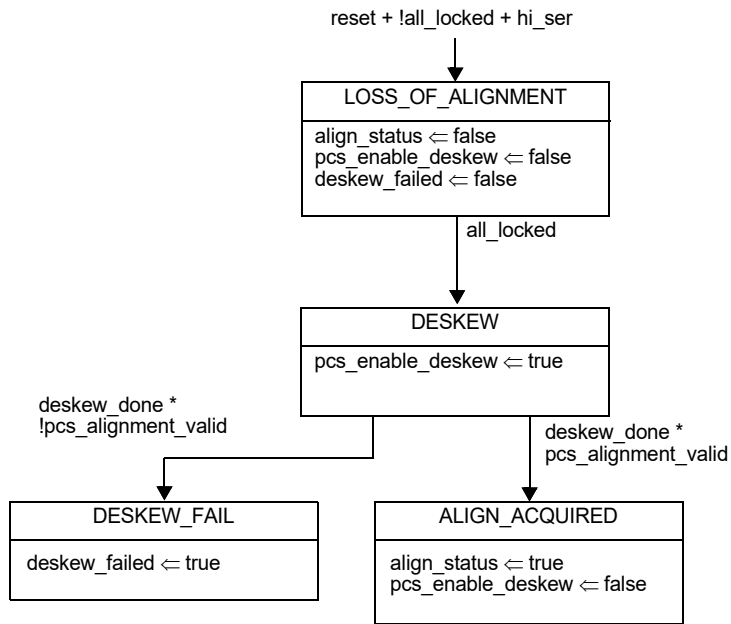


Figure 175–8—PCS synchronization state diagram

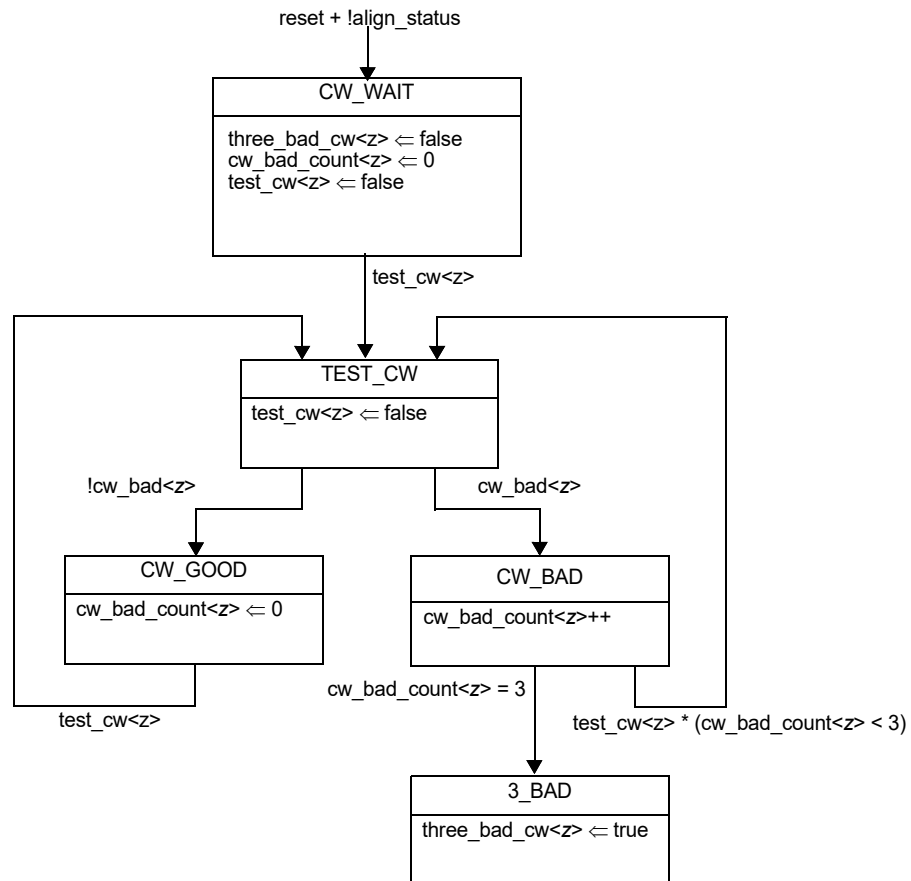


Figure 175-9—PCS codeword monitor state diagram

175.3 FEC Degrad

The FEC degrade feature provides the ability to detect degrade conditions at the RS-FEC decoder using FEC degrade detection and to propagate all detected FEC degrade conditions using FEC degrade signaling. The propagation of FEC degrade signals across PCS and XS sublayers is described in 116.6.

FEC degrade detection is specified in 119.2.5.3 and 175.2.5.3. FEC degrade detection is optional.

FEC degrade signaling using the PCS alignment marker status field is specified for the PCS in 175.2.4.6. FEC degrade signaling across the PCS service interface is specified in 175.1.4.1. FEC degrade signaling is required.

175.4 Loopback

Loopback is identical to that specified in 119.4.

The loopback function is enabled when the PCS control variable loopback is set to 1.

175.5 Delay constraints

The maximum delay contributed by the 1.6TBASE-R PCS (sum of transmit and receive delays at one end of the link) shall be no more than 1280000 bit times, equivalent to 2500 pause_quanta or 800 ns. A description of overall system delay constraints and the definition of bit times and pause_quanta can be found in 174.4 and its references.

175.6 Path data delay for time synchronization

The PCS sublayer optionally provides path data delay information for time synchronization (see Clause 90).

The ability to report PCS path data delays is indicated by the PCS_delay_ns_TX_ability, PCS_delay_subns_TX_ability, PCS_delay_ns_RX_ability, and PCS_delay_subns_RX_ability variables (see Table 175–7).

The PCS lane distribution can cause significant delay variations as described in 90.7.1. When the PCS_timesync_multilane_ability variable is asserted, the maximum delay value through the PCS is allocated to the transmit path and the minimum delay value to the receive path.

The transmit and receive path data delays are reported as the delay incurred by the first bit of a set of four interleaved RS-FEC codewords. Additionally, the receive path data delay is reported as if the PCS lanes arrive without any skew (see 90.7.3) which is equivalent to the first bit of a set of four interleaved RS-FEC codewords being present in the last arriving PCS lane.

Four separate delays are reported in the following eight path data delay status variables:

- PCS_delay_ns_TX_max and PCS_delay_subns_TX_max
- PCS_delay_ns_TX_min and PCS_delay_subns_TX_min
- PCS_delay_ns_RX_max and PCS_delay_subns_RX_max
- PCS_delay_ns_RX_min and PCS_delay_subns_RX_min

175.7 Auto-Negotiation

The following requirements apply to a PCS used with a 1.6TBASE-CR8 or 1.6TBASE-KR8 PMD where support for the auto-negotiation process defined in Clause 73 is mandatory. The PCS shall support the AN_LINK.indication(link_status) primitive (see 73.9). The parameter link_status shall take one of the values FAIL, IN_PROGRESS, or OK, according to Table 175–5. The primitive shall be generated when the value of link_status changes.

Table 175–5—AN_LINK.indication(link_status) generation

reset + restart_lock	align_status	link_status
true	don't care	FAIL
false	false	IN_PROGRESS
	true	OK

The PCS variables reset, restart_lock, and align_status are defined in 175.2.6.2.2.

The following PCS status variable is defined for link_status:

in_progress_indication

Boolean variable that is true when the link_status parameter of AN_LINK.indication has the value IN_PROGRESS and is false otherwise.

175.8 Management variables

PCS control and status variables intended to be accessible via a management system are listed in Table 175–6 and Table 175–7.

If the MDIO interface (see Clause 45) is implemented, PCS control and status variables shall be mapped to MDIO registers as shown in Table 175–6 and Table 175–7.

If the MDIO interface is not implemented, an alternate mechanism to access management variables shall be provided.

Table 175–6—PCS control variables and MDIO mapping

Control variable	Variable reference	MDIO register/bit number	MDIO register/bit reference
PCS_reset	175.2.6.2.2	3.0.15	45.2.3.1
loopback	175.4	3.0.14	45.2.3.1
tx_test_mode	175.2.4.11	3.42.3	45.2.3.19
FEC_bypass_indication_enable	175.2.5.3	3.800.1	45.2.3.60
FEC_degraded_SER_enable	175.2.5.3	3.800.2	45.2.3.60
FEC_degraded_SER_activate_threshold	175.2.5.3	3.806, 3.807	45.2.3.64
FEC_degraded_SER_deactivate_threshold	175.2.5.3	3.808, 3.809	45.2.3.65
FEC_degraded_SER_interval	175.2.5.3	3.810, 3.811	45.2.3.66

Table 175–7—PCS status variables and MDIO mapping

Status variable	Variable reference	MDIO register/bit number	MDIO register/bit reference
PCS_status	175.2.6.2.2	3.32.12	45.2.3.15.1
align_status	175.2.6.2.2 Figure 175–8	3.50.12	45.2.3.23.1
amps_lock<15:0>	175.2.6.2.2 Figure 119–12	3.53.7:0, 3.52.7:0	45.2.3.26, 45.2.3.25
FEC_cw_counter	175.2.5.3	3.300 to 3.302	45.2.3.48a
FEC_codeword_error_bin_<1:15>	175.2.5.3	3.340 to 3.369	45.2.3.48b
pcs_lane_mapping<0:15>	175.2.6.2.2 Figure 119–12	3.400 to 3.415	45.2.3.49

Table 175–7—PCS status variables and MDIO mapping (*continued*)

Status variable	Variable reference	MDIO register/bit number	MDIO register/bit reference
FEC_symbol_error_counter_<0:15>	175.2.5.3	3.600 to 3.631	45.2.3.58
FEC_bypass_indication_ability	119.2.5.3	3.801.1	45.2.3.61.6
hi_ser	175.2.5.3 175.2.6.2.2	3.801.2	45.2.3.61.5
FEC_degraded_SER_ability	119.2.5.3	3.801.3	45.2.3.61.4
FEC_degraded_SER	175.2.5.3	3.801.4	45.2.3.61.3
rx_rm_degraded	175.2.5.5	3.801.5	45.2.3.61.2
rx_local_degraded	175.2.5.5	3.801.6	45.2.3.61.1
FEC_corrected_cw_counter	175.2.5.3	3.802, 3.803	45.2.3.62
FEC_uncorrected_cw_counter	175.2.5.3	3.804, 3.805	45.2.3.63
PCS_delay_ns_TX_max	175.6	3.1801, 3.1802	45.2.3.68
PCS_delay_subns_TX_max	175.6	3.1809	45.2.3.68
PCS_delay_ns_TX_min	175.6	3.1803, 3.1804	45.2.3.68
PCS_delay_subns_TX_min	175.6	3.1810	45.2.3.68
PCS_delay_ns_RX_max	175.6	3.1805, 3.1806	45.2.3.69
PCS_delay_subns_RX_max	175.6	3.1811	45.2.3.69
PCS_delay_ns_RX_min	175.6	3.1807, 3.1808	45.2.3.69
PCS_delay_subns_RX_min	175.6	3.1812	45.2.3.69
PCS_timesync_multilane_ability	175.6	3.1800.11	45.2.3.67
PCS_delay_ns_RX_ability	175.6	3.1800.0	45.2.3.67
PCS_delay_ns_TX_ability	175.6	3.1800.1	45.2.3.67
PCS_delay_subns_RX_ability	175.6	3.1800.2	45.2.3.67
PCS_delay_subns_TX_ability	175.6	3.1800.3	45.2.3.67
in_progress_indication	175.7	3.1.3	45.2.3.2.6b

175.9 Protocol implementation conformance statement (PICS) proforma for Clause 175, Physical Coding Sublayer (PCS), type 1.6TBASE-R⁵

175.9.1 Introduction

The supplier of a protocol implementation that is claimed to conform to Clause 175, Physical Coding Sublayer (PCS), type 1.6TBASE-R, shall complete the following protocol implementation conformance statement (PICS) proforma.

A detailed description of the symbols used in the PICS proforma, along with instructions for completing the PICS proforma, can be found in Clause 21.

175.9.2 Identification

175.9.2.1 Implementation identification

Supplier ¹	
Contact point for inquiries about the PICS ¹	
Implementation Name(s) and Version(s) ^{1,3}	
Other information necessary for full identification—e.g., name(s) and version(s) for machines and/or operating systems; System Name(s) ²	
NOTE 1—Required for all implementations. NOTE 2—May be completed as appropriate in meeting the requirements for the identification. NOTE 3—The terms Name and Version should be interpreted appropriately to correspond with a supplier's terminology (e.g., Type, Series, Model).	

175.9.2.2 Protocol summary

Identification of protocol standard	IEEE Std 802.3dj-202x, Clause 175, Physical Coding Sublayer (PCS), type 1.6TBASE-R
Identification of amendments and corrigenda to this PICS proforma that have been completed as part of this PICS	
Have any Exception items been required? No [] Yes [] (See Clause 21; the answer Yes means that the implementation does not conform to IEEE Std 802.3dj-202x.)	

Date of Statement	
-------------------	--

⁵Copyright release for PICS proformas: Users of this standard may freely reproduce the PICS proforma in this subclause so that it can be used for its intended purpose and may further publish the completed PICS.

175.9.3 Major capabilities/options

Item	Feature	Subclause	Value/Comment	Status	Support
MII	1.6TMII logical interface is supported	175.1.4.1		O	Yes [] No []
*MD	MDIO	175.8		O	Yes [] No []
*FDD	Support for FEC degrade detection	175.2.5.3		O	Yes [] No []

175.9.4 PICS proforma tables for Physical Coding Sublayer (PCS), type 1.6TBASE-R

175.9.4.1 Transmit function

Item	Feature	Subclause	Value/Comment	Status	Support
*SE	Uses stateless encoder	175.2.4.1		O	Yes [] No []
TF1	Transmit 64B/66B encoder uses state-diagram encoder defined in 119.2.4.1.1	175.2.4.1		!SE:M	Yes [] N/A []
TF2	Transmit 64B/66B encoder uses stateless encoder defined in 119.2.4.1.2.	175.2.4.1		SE:M	Yes [] N/A []
TF3	64B/66B to 256B/257B transcoder	175.2.4.3		M	Yes []
TF4	Scrambler	175.2.4.5		M	Yes []
TF5	Pad value	175.2.4.6		M	Yes []
TF6	Alignment marker insertion	175.2.4.6		M	Yes []
TF7	FEC degrade signaling	175.2.4.6		M	Yes []
TF8	Pre-FEC distribution	175.2.4.7		M	Yes []
TF9	Reed-Solomon encoder	175.2.4.8		M	Yes []
TF10	Symbol distribution	175.2.4.9		M	Yes []
TF11	Transmission bit ordering	175.2.4.10		M	Yes []
TF12	Scrambled idle transmit test-pattern generator	175.2.4.11		M	Yes []

175.9.4.2 Receive function

Item	Feature	Subclause	Value/Comment	Status	Support
*BI	Bypass indication supported	175.2.5.3		O	Yes [] No []
*SD	Uses stateless decoder	175.2.5.9		O	Yes [] No []
RF1	Alignment marker lock	175.2.5.1		M	Yes []
RF2	PCS lane number is captured to a management variable	175.2.6.3		M	Yes []
RF3	Deskew	175.2.5.1		M	Yes []
RF4	Lane reorder and deinterleave	175.2.5.2		M	Yes []
RF5	Reed-Solomon decoder corrects any combination of up to $t=15$ symbol errors in a codeword	175.2.5.3		M	Yes []
RF6	Reed-Solomon decoder is capable of indicating when a codeword was not corrected.	175.2.5.3		M	Yes []
RF7	Error monitoring and assertion of hi_ser, while error indication is bypassed	175.2.5.3		BI:M	Yes [] N/A []
RF8	Detect FEC degrade	175.2.5.3		FDD:M	Yes [] N/A []
RF9	FEC degrade signaling on PCS service interface	175.1.4.1		M	Yes []
RF10	Alignment marker removal	175.2.5.5		M	Yes []
RF11	Descrambler	175.2.5.6		M	Yes []
RF12	256B/257B to 64B/66B transcoder	175.2.5.8		M	Yes []
RF13	Receive 64B/66B decoder uses state-diagram decoder defined in 119.2.5.8.1	175.2.5.9		!SD:M	Yes [] N/A []
RF14	Receive 64B/66B decoder uses stateless decoder defined in 119.2.5.8.2	175.2.5.9		SD:M	Yes [] N/A []

175.9.4.3 64B/66B coding rules

Item	Feature	Subclause	Value/Comment	Status	Support
C1	Encoder (and ENCODE function) implements the code as specified	175.2.3, 175.2.6.2.3		M	Yes []
C2	Decoder (and DECODE function) implements the code as specified	175.2.3, 175.2.6.2.3		M	Yes []
C3	Only valid block types are transmitted	175.2.3		M	Yes []
C4	Invalid block types are treated as an error	175.2.3		M	Yes []
C5	Only valid control characters are transmitted	175.2.3		M	Yes []
C6	Invalid control characters are treated as an error	175.2.3		M	Yes []
C7	Idles do not interrupt data	175.2.3		M	Yes []
C8	IDLE control code insertion and deletion	175.2.3	Insertion or deletion in groups of 8 /I/s	M	Yes []
C9	Sequence ordered set deletion	175.2.3	Only one whole ordered set of two consecutive sequence ordered sets may be deleted	M	Yes []

175.9.4.4 Loopback

Item	Feature	Subclause	Value/Comment	Status	Support
L1	Supports loopback	175.4		M	Yes []
L2	When in loopback, transmits what it receives from the 1.6TMII	175.4		M	Yes []
L3	When in loopback, ignore all data presented by the sublayer below the PCS.	175.4		M	Yes []

175.9.4.5 Delay constraints

Item	Feature	Subclause	Value/Comment	Status	Support
TIM1	PCS delay constraint	175.5		M	Yes []

175.9.4.6 Time synchronization

Item	Feature	Subclause	Value/Comment	Status	Support
*TS	Support for time synchronization	175.6		O	Yes [] No []
TS1	Path data delays	175.6	Path data delays reported as if DDMP is at start of a set of four interleaved RS-FEC codewords.	TS:M	Yes [] N/A []
TS2	Nanosecond resolution	175.6	Path data delays reported with nanosecond or better resolution.	TS:M	Yes [] N/A []
TS3	Sub-nanosecond resolution	175.6	Path data delays reported with sub-nanosecond resolution.	TS:O	Yes [] No [] N/A []

175.9.4.7 Auto-Negotiation

Item	Feature	Subclause	Value/Comment	Status	Support
*AN	Support for use with a 1.6TBASE-CR8 or 1.6TBASE-KR8 PMD	175.7	AN technology dependent interface described in Clause 73	O	Yes [] No []
AN1	AN_LINK.indication primitive	175.7	Support of the AN_LINK.indication(link_status) primitive	AN:M	Yes [] N/A []
AN2	link_status parameter	175.7	Takes the value OK, IN_PROGRESS or FAIL.	AN:M	Yes [] N/A []
AN3	Generation of AN_LINK.indication primitive	175.7	Generated when the value of link_status changes	AN:M	Yes [] N/A []

175.9.4.8 Management

Item	Feature	Subclause	Value/Comment	Status	Support
M1	Alternate access to management variables is provided	175.8		!MD:M	Yes [] N/A []
M2	Mapping of MDIO control bits and MDIO status bits	175.8		MD:M	Yes [] N/A []